AttendEase — Notification System (Complete Documentation)

Date: March 2026 | **Status:** Planning / Pre-Development Phase

---

1. What Sends Notifications in AttendEase?

There are **6 triggers** in your app that need notifications:

Trigger	Who Gets It	Type
Forgot to check out	Employee	Push
Leave approved/rejected	Employee	Push
Regularisation approved/rejected	Employee	Push
New employee added	Employee	Push
Late / absent marked	Employee	Push
Leave request submitted	Admin/Manager	Push + In-App

---

2. The Players Involved

Every notification in AttendEase passes through 4 layers. Each layer has exactly one job.

```
mermaid
```

```
flowchart TD
  A[Node.js Backend\nDecides what to send and when] --> B[Bull Queue\nSchedules delayed jobs\ne.g. forgot checkout afte
  A --> C[sendPushNotification function\nYour central notification dispatcher]
  B --> C
  C --> D[Firebase FCM\nGoogle's free push delivery service]
  D --> E[Employee Phone\nAndroid via GCM / iOS via APNs]
  C --> F[notifications table\nIn-app bell history]
  C --> G[Socket.io\nReal-time badge update when app is open]

  style A fill:#1e293b,color:#fff
  style B fill:#7c3aed,color:#fff
  style C fill:#0f766e,color:#fff
  style D fill:#ea580c,color:#fff
  style E fill:#15803d,color:#fff
  style F fill:#1d4ed8,color:#fff
  style G fill:#b45309,color:#fff
```

- **Bull Queue** → decides *when* to send (the alarm clock)
- **FCM** → decides *how* to deliver (the courier)
- **Socket.io** → real-time bell badge update while app is open
- **notifications table** → stores every notification for in-app history

---

3. How FCM Works (Plain English)

FCM (Firebase Cloud Messaging) is Google's free service that delivers push notifications to Android and iOS devices. You cannot send a notification directly to a phone — Apple and Google control what reaches their devices. FCM is the only route.

```mermaid
sequenceDiagram
  participant BE as Your Backend
  participant FCM as Firebase FCM
  participant APNs as Apple APNs (iOS only)
  participant Phone as Employee Phone

  BE->>FCM: Send notification with FCM token
  FCM->>FCM: Route: Android or iOS?
  FCM->>Phone: Android → direct delivery
  FCM->>APNs: iOS → route via Apple
  APNs->>Phone: iOS delivery
  Phone-->>BE: Delivery receipt (success/fail)
```

Step 1 — App opens for first time: The React Native app calls Firebase SDK. Firebase generates a unique FCM Token (a long string) for this device + app combination. Your app sends this token to your backend. Backend stores it in the `device_tokens` table.

Step 2 — You want to send a notification: Your Node.js backend calls Firebase's HTTP API with the FCM token (who to send to), and the title + body + data payload.

Step 3 — Firebase delivers it: Firebase routes it to Google's servers (Android) or Apple's APNs (iOS). It lands on the employee's phone even if the app is closed.

---

4. FCM Token — Automatic Generation

Short answer: Firebase generates it automatically. You just catch it and save it.

What Actually Happens Step by Step

```mermaid
sequenceDiagram
    participant App as React Native App
    participant FB as Firebase SDK
    participant G as Google Servers
    participant BE as Your Backend
    participant DB as device_tokens table

    App->>FB: App opens, SDK initializes automatically
    FB->>G: Contact Google servers in background
    G-->>FB: Generate unique FCM Token for this device+app
    FB-->>App: Token ready via getToken() callback
    App->>BE: POST /api/v1/device-tokens { fcm_token }
    BE->>DB: UPSERT token (insert if new, update timestamp if exists)
    Note over App,DB: Zero manual token generation — Firebase handles everything
```

You write zero token generation logic. Firebase handles all of that. Your only job is to receive and store it.

React Native Code — Token Registration

```javascript
```

```
// Inside App.js — runs once when app loads

import messaging from '@react-native-firebase/messaging';

async function registerDeviceToken() {

  // Step 1: Ask notification permission (iOS requires this, Android 13+ too)
  const permission = await messaging().requestPermission();

  if (permission === messaging.AuthorizationStatus.AUTHORIZED) {

    // Step 2: Get the token — Firebase already generated it, just fetch it
    const fcmToken = await messaging().getToken();

    // Step 3: Send to your backend and save in device_tokens table
    await api.post('/device-tokens', { fcm_token: fcmToken });
  }
}

// Step 4: Listen for token REFRESH (token can change anytime)
messaging().onTokenRefresh(async (newToken) => {
  await api.post('/device-tokens', { fcm_token: newToken });
  // Your backend upserts — old token deactivated, new one saved
});
```

When Does the Token Change?

The token is **not permanent**. This catches most developers off guard.

Situation	Token Changes?
App first install	✅ New token generated
Employee uninstalls + reinstalls	✅ New token generated
Employee clears app data	✅ New token generated
Employee gets a new phone	✅ New token generated
App update	❌ Token stays same
Phone restarts	❌ Token stays same
Employee logs out your app	❌ Token stays same (Firebase doesn't know)

This is why `onTokenRefresh` listener is **mandatory**. Without it, notifications silently stop working after reinstall and nobody knows why.

Backend Code — Saving the Token

```javascript
// POST /api/v1/device-tokens
async function saveDeviceToken(req, res) {
  const { fcm_token } = req.body;
  const { emp_id, org_id } = req.user; // from JWT

  // UPSERT — insert if new, update timestamp if exists
  await db.query(`
    INSERT INTO device_tokens (org_id, emp_id, fcm_token, platform, is_active)
    VALUES ($1, $2, $3, $4, TRUE)
    ON CONFLICT (emp_id, fcm_token)
    DO UPDATE SET is_active = TRUE, updated_at = NOW()
  `, [org_id, emp_id, fcm_token, platform]);

  res.json({ success: true });
}
```

`ON CONFLICT` handles everything. If same token comes again (app restart), it just updates the timestamp. No duplicates.

On Logout — Deregister the Token

```javascript
// When employee logs out of your app:
async function logout() {
  const fcmToken = await messaging().getToken();

  // Tell your backend to deactivate this token
  await api.delete('/device-tokens', { fcm_token: fcmToken });

  // Clear JWT, navigate to login
}
```

```javascript
// Backend DELETE /device-tokens
await db.query(
  `UPDATE device_tokens SET is_active = FALSE WHERE fcm_token = $1`,
  [fcm_token]
);
```

Why deactivate on logout? If an employee logs out and someone else logs into the same phone (shared device scenario), you don't want the old employee getting the new employee's notifications.

Full Token Lifecycle

```mermaid
flowchart TD
  A[App Install] -->|Firebase generates token| B[Token saved in DB\nis_active: true]
  B --> C{What happens next?}
  C -->|App restarts| D[Same token returned\nUPSERT updates timestamp]
  C -->|Token refreshes| E[New token from Firebase\nOld row deactivated\nNew row inserted]
  C -->|Uninstall| F[Firebase knows token is dead\nReturns 404 on send\nBackend sets is_active: false]
  C -->|Logout| G[Your app tells backend\nis_active: false immediately]
  C -->|Login again| H[getToken called again\nToken saved or reactivated]
  D --> C
  H --> C

  style A fill:#15803d,color:#fff
  style B fill:#1d4ed8,color:#fff
  style F fill:#dc2626,color:#fff
  style G fill:#dc2626,color:#fff
```

---

5. Bull Queue vs FCM — They Are Different Things

A common confusion: Bull Queue and FCM look similar because they work together. But they are completely different tools.

	Bull Queue	**FCM**
Job	Decides WHEN to send	Decides HOW to deliver
Built on	Redis	Google infrastructure
Has scheduling?	✅ Yes — delay jobs by hours	❌ No — delivers immediately
Survives server restart?	✅ Yes — stored in Redis	N/A
Cost	$0 (uses your Redis)	$0 forever

The Analogy

Think of it like a courier system:

```
You (Backend)      → decide WHEN to send the package
Bull Queue         → the WAREHOUSE that holds it until dispatch time
FCM                → the COURIER (Dunzo / Swiggy Instamart) that delivers it
Employee's Phone   → the DOOR where it gets delivered
```

FCM has absolutely no concept of time or scheduling. You call FCM and it delivers **immediately**. That's its only job.

The Code Makes It Obvious

```javascript
// Bull Queue worker — wakes up at 5 PM
checkoutReminderQueue.process(async (job) => {

  // Bull Queue's job is DONE here.
  // It just woke up at the right time.

  // Now YOU call FCM:
  await firebase.messaging().send({
    token: fcm_token,
    notification: {
      title: "Don't forget to check out!"
    }
  });
  // FCM's job starts HERE — just deliver this, right now.
});
```

What About RabbitMQ?

RabbitMQ is a **message broker** — it sits between microservices and routes messages between them. It is not needed for AttendEase right now because:

- You are running a **monolith** — all services are in the same codebase
- In a monolith, functions call each other directly — no broker needed
- RabbitMQ only makes sense when services are **separate deployed applications**

When you would add RabbitMQ (future, at 50,000+ employees when splitting to microservices):

```mermaid
```

```
flowchart LR
  A[Attendance Service] --> R[RabbitMQ]
  R --> B[Notification Service]
  R --> C[Payroll Service]
  R --> D[Analytics Service]

  style R fill:#ea580c,color:#fff
```

For now: Bull Queue + direct function calls is the right choice.

---

6. Why Not setTimeout Instead of Bull Queue?

What Looks Like It Would Work

```javascript
// Seems fine, right?
app.post('/check-in', async (req, res) => {
  await saveAttendance(emp_id);

  setTimeout(async () => {
    await sendCheckoutReminder(emp_id);
  }, 8 * 60 * 60 * 1000); // 8 hours later

  res.json({ success: true });
});
```

Looks clean. Actually a disaster in production.

Problem 1 — Server Restarts Kill Everything

```
9:00 AM  → Employee checks in
9:00 AM  → setTimeout scheduled for 5:00 PM
           (stored only in Node.js RAM)

2:00 PM  → AWS EC2 restarts (deployment, crash, memory issue)
           setTimeout is GONE. Lives only in RAM.

5:00 PM  → No reminder sent. Ever.
           Employee never checks out.
           Attendance marked absent.
           Employee raises complaint.
           You have no idea what happened.
```

Bull Queue stores jobs in **Redis (disk-backed)**. Server restart = jobs survive.

Problem 2 — 10,000 Timers Sitting in RAM

```
10,000 employees check in at 9 AM
→ 10,000 simultaneous setTimeouts created
→ All sitting in Node.js process memory
→ Each holding closures and data

Node.js process memory explodes.
Server slows down. Some timers drop silently.
```

Bull Queue stores jobs in Redis, **outside your Node.js process**. App RAM stays clean.

Problem 3 — No Retry on Failure

```javascript
setTimeout(async () => {
  await sendCheckoutReminder(emp_id);
  // What if FCM is down for 2 minutes?
  // What if DB connection dropped?
  // Job just... fails silently.
  // No retry. No log. No alert.
}, delay);
```

Bull Queue has built-in retry:

```javascript
queue.add(data, {
  attempts: 3,       // retry 3 times
  backoff: 5000      // wait 5 seconds between retries
});
```

Problem 4 — You Can't See Anything

```
With setTimeout:
❌ Can't see how many jobs are waiting
❌ Can't see which jobs failed
❌ Can't cancel a job (employee already checked out)
❌ Can't pause all jobs during maintenance
❌ No dashboard, no visibility

With Bull Queue:
✅ Bull Board dashboard shows every job
✅ Failed jobs logged with full error
```

✅ Cancel a specific job by ID
✅ Pause entire queue during deployment

Problem 5 — Can't Cancel a setTimeout

```javascript
// When employee checks out early, you need to cancel the reminder.
// With setTimeout you must do this ugly thing:

const timers = new Map(); // stored in memory — dies on restart

const timer = setTimeout(...);
timers.set(attendance_id, timer);

// On checkout:
clearTimeout(timers.get(attendance_id));
timers.delete(attendance_id);
// This Map also dies on server restart. Back to Problem 1.
```

With Bull Queue:

```javascript
// Clean, survives restarts:
const job = await queue.add(data, { delay });
await job.remove(); // cancel anytime, from anywhere
```

Comparison Table

Situation	setTimeout	Bull Queue
Server restarts	❌ All jobs lost	✅ All jobs survive
10,000 jobs	❌ RAM overload	✅ Stored in Redis
Job fails	❌ Silent failure	✅ Auto retry
Cancel a job	❌ Must track manually	✅ `job.remove()`
See pending jobs	❌ Invisible	✅ Dashboard
Multiple servers	❌ Impossible	✅ Built-in

7. Forgot Checkout Flow (Most Complex — Bull Queue)

This is the most important delayed notification in AttendEase.

```mermaid
sequenceDiagram
    participant E as Employee
    participant App as React Native App
    participant BE as Backend
    participant BQ as Bull Queue (Redis)
    participant FCM as Firebase FCM
    participant Phone as Employee Phone

    E->>App: Taps Check In at 9:00 AM
    App->>BE: POST /attendance/check-in
    BE->>BE: Save attendance record
    BE->>BQ: Schedule job: fire at 5:30 PM\n(shift_end + 30 min)
    BE-->>App: Check-in confirmed

    Note over BQ: 8 hours pass...

    BQ->>BE: Job fires at 5:30 PM
    BE->>BE: Query DB: is check_out_time NULL?
    alt Employee forgot to check out
        BE->>FCM: Send notification to FCM token
        FCM->>Phone: "You haven't checked out yet"
        E->>App: Taps notification
        App->>App: Deep link opens Checkout screen
        E->>App: Taps Check Out
        App->>BE: POST /attendance/check-out
    else Employee already checked out
        BE->>BQ: Job does nothing, exits cleanly
    end
```

The Bull Queue Code

```javascript
```

```javascript
// When employee checks in:
await checkoutReminderQueue.add(
  { emp_id, attendance_id, org_id },
  { delay: msUntilShiftEnd + 30 * 60 * 1000 } // fires after shift ends + 30 min
);

// The worker — runs when delay expires:
checkoutReminderQueue.process(async (job) => {
  const { emp_id, attendance_id } = job.data;

  const attendance = await db.query(
    'SELECT check_out_time FROM attendance WHERE id = $1',
    [attendance_id]
  );

  if (!attendance.check_out_time) {
    await sendPushNotification(emp_id, 'checkout_reminder', {
      title: "Don't forget to check out!",
      body: "Tap here to mark your check-out.",
      action_url: `attendease://checkout/${attendance_id}`
    });
  }
  // If already checked out → job does nothing, exits cleanly
});
```

---

8. The Core sendPushNotification() Function

This is the single function every part of your backend calls. All notification logic passes through here.

```javascript
```

```
async function sendPushNotification(emp_id, type, { title, body, data, action_url }) {

  // Step 1: Check employee notification preference
  const employee = await db.query(
    `SELECT notification_preferences, org_id FROM employees WHERE id = $1`,
    [emp_id]
  );
  const { notification_preferences: prefs, org_id } = employee.rows[0];
  if (prefs[type] === false) return; // employee turned this off

  // Step 2: Check Redis rate limiting cooldown
  const cooldownKey = `notif_cooldown:${emp_id}:${type}`;
  const alreadySent = await redis.get(cooldownKey);
  if (alreadySent) return; // cooldown active, skip

  // Step 3: Get all active tokens for this employee
  const tokens = await db.query(
    `SELECT fcm_token, platform FROM device_tokens
     WHERE emp_id = $1 AND is_active = TRUE`,
    [emp_id]
  );

  if (tokens.rows.length === 0) return; // no device registered

  // Step 4: Set cooldown before sending (not after)
  const cooldowns = {
    checkout_reminder: 60 * 60,    // 1 hour
    late_marked: 4 * 60 * 60,      // 4 hours
    leave_approved: 5 * 60,        // 5 minutes
    announcements: 10 * 60,        // 10 minutes
  };
  await redis.setex(cooldownKey, cooldowns[type] || 300, '1');

  // Step 5: Send to each device (employee may have phone + tablet)
  const results = await Promise.allSettled(
    tokens.rows.map(({ fcm_token }) =>
      firebase.messaging().send({
        token: fcm_token,
        notification: { title, body },
        data: { action_url, sent_at: String(Date.now()), ...data },
        android: { priority: 'high' },
        apns: { payload: { aps: { sound: 'default', 'content-available': 1 } } }
      })
    )
  );
```

```
  // Step 6: Handle dead tokens
  results.forEach((result, i) => {
    if (result.status === 'rejected' &&
        result.reason.code === 'messaging/registration-token-not-registered') {
      db.query(
        `UPDATE device_tokens SET is_active = FALSE WHERE fcm_token = $1`,
        [tokens.rows[i].fcm_token]
      );
    }
  });

  // Step 7: Write to notifications table (for in-app bell)
  await db.query(
    `INSERT INTO notifications (org_id, emp_id, type, title, body, data, action_url)
     VALUES ($1, $2, $3, $4, $5, $6, $7)`,
    [org_id, emp_id, type, title, body, JSON.stringify(data), action_url]
  );

  // Step 8: Emit via Socket.io for real-time badge update
  io.to(`org_${org_id}_emp_${emp_id}`).emit('new_notification', {
    title, body, action_url
  });
}
```

---

9. Foreground / Background / Killed App States

The same FCM message behaves differently depending on the app state. This is the most misunderstood part of React Native notifications.

```mermaid
```

```
flowchart TD
  N[FCM Notification Arrives] --> Q{App State?}
  Q -->|Foreground| F[App is open\nEmployee looking at it]
  Q -->|Background| B[App minimized\nRunning behind]
  Q -->|Killed| K[App swiped away\nNot running at all]

  F --> F1[❌ System does NOT show\nnotification automatically]
  F --> F2[✅ Your onMessage code runs]
  F2 --> F3[You must show manual\nin-app toast/banner]

  B --> B1[✅ System shows notification\non lock screen]
  B --> B2[✅ onNotificationOpenedApp fires\nwhen employee taps]

  K --> K1[✅ System shows notification\on lock screen]
  K --> K2[✅ getInitialNotification fires\nonce on cold start]

  style F fill:#b45309,color:#fff
  style B fill:#1d4ed8,color:#fff
  style K fill:#dc2626,color:#fff
```

State 1: Foreground — App is Open

System does NOT show the notification automatically. You must show something manually.

```javascript
messaging().onMessage(async (remoteMessage) => {
  // App is open — show an in-app toast/banner manually
  showInAppBanner({
    title: remoteMessage.notification.title,
    body: remoteMessage.notification.body,
    onTap: () => navigateTo(remoteMessage.data.action_url)
  });

  // Also update bell badge count
  setBadgeCount(prev => prev + 1);
});
```

State 2: Background — App is Minimized

System already showed the notification on the lock screen. This fires only when the employee actually **taps** it.

```javascript
```

```
messaging().onNotificationOpenedApp((remoteMessage) => {
  // App was in background, employee tapped notification
  navigateTo(remoteMessage.data.action_url);
});
```

State 3: Killed — App Was Closed

App was dead. Employee tapped notification. App cold starts. Check **once** on app launch.

```javascript
useEffect(() => {
  messaging()
    .getInitialNotification()
    .then((remoteMessage) => {
      if (remoteMessage) {
        // App was opened by tapping a notification
        navigateTo(remoteMessage.data.action_url);
      }
    });
}, []);
```

Behaviour Summary

State	Notification Shows?	Your Code Runs?	You Must Handle
Foreground	❌ Not automatically	✅ Yes	Show manual in-app banner
Background	✅ System shows it	✅ On tap	Handle tap navigation
Killed	✅ System shows it	✅ On tap only	Handle cold start navigation

---

10. Deep Linking — action_url Opens the Right Screen

When employee taps a notification, they must land on the **exact relevant screen**, not just the home screen.

Your action_url Structure

```
attendease://leave/123        → Leave detail screen for leave ID 123
attendease://checkout/456     → Checkout screen for attendance ID 456
attendease://regularise/789   → Regularisation screen
attendease://attendance       → Attendance history screen
```

Step 1 — Register Deep Link Scheme in Android

In `AndroidManifest.xml`:

```xml
<intent-filter>
  <action android:name="android.intent.action.VIEW" />
  <data android:scheme="attendease" />
</intent-filter>
```

Step 2 — Register Deep Link Scheme in iOS

In `Info.plist`:

```xml
<key>CFBundleURLSchemes</key>
<array>
  <string>attendease</string>
</array>
```

Step 3 — Central Navigation Handler

Called from all 3 app states (foreground, background, killed):

```javascript
```

```
function navigateTo(action_url) {
  if (!action_url) return;

  // Parse: "attendease://leave/123"
  const path = action_url.replace('attendease://', '');
  const [screen, id] = path.split('/');

  switch (screen) {
    case 'leave':
      navigation.navigate('LeaveDetail', { leave_id: id });
      break;
    case 'checkout':
      navigation.navigate('CheckOut', { attendance_id: id });
      break;
    case 'regularise':
      navigation.navigate('Regularisation', { attendance_id: id });
      break;
    case 'attendance':
      navigation.navigate('AttendanceHistory');
      break;
    default:
      navigation.navigate('Home');
  }
}
```

---

11. Notification Preferences

Employees should control what they get notified about. Spamming them kills engagement and gets the app disabled.

DB Design — JSONB on employees table

```sql
ALTER TABLE employees
ADD COLUMN notification_preferences JSONB DEFAULT '{
  "leave_approved": true,
  "leave_rejected": true,
  "late_marked": true,
  "absent_marked": true,
  "regularisation_approved": true,
  "checkout_reminder": true,
  "announcements": true
}';
```

Using JSONB means **no migration needed** when adding new notification types — just add a new key.

Backend Checks Preference Before Sending

```javascript
async function sendPushNotification(emp_id, type, payload) {

  const employee = await db.query(
    `SELECT notification_preferences FROM employees WHERE id = $1`,
    [emp_id]
  );

  const prefs = employee.rows[0].notification_preferences;

  // If employee turned this off — stop here, don't send
  if (prefs[type] === false) return;

  // Otherwise proceed with FCM send
}
```

Settings Screen API

Employee toggles a setting → React Native calls:

```
PATCH /api/v1/employees/me/notification-preferences
Body: { "late_marked": false }
```

---

12. Bulk Notifications — Admin Broadcasting

Admin wants to send a message to 500 employees in a department. You **cannot loop 500 individual FCM calls**.

Why Looping Is Wrong

```javascript
// WRONG — never do this
employees.forEach(async (emp) => {
  await firebase.messaging().send({ token: emp.fcm_token, ... });
});
// 500 sequential API calls
// FCM rate limits you
// Takes minutes to complete
// One failure can break the loop
```

FCM Multicast — The Correct Way

FCM allows **500 tokens per single API call**:

```javascript
async function sendBulkNotification(org_id, department_id, { title, body }) {

  // Get all tokens for the department
  const tokens = await db.query(`
    SELECT dt.fcm_token
    FROM device_tokens dt
    JOIN employees e ON e.id = dt.emp_id
    WHERE e.org_id = $1
    AND e.department_id = $2
    AND dt.is_active = TRUE
  `, [org_id, department_id]);

  const allTokens = tokens.rows.map(r => r.fcm_token);

  // FCM max 500 per call — chunk them
  const chunks = chunkArray(allTokens, 500);

  for (const chunk of chunks) {
    await firebase.messaging().sendEachForMulticast({
      tokens: chunk,
      notification: { title, body }
    });
  }
}

function chunkArray(arr, size) {
  return Array.from(
    { length: Math.ceil(arr.length / size) },
    (_, i) => arr.slice(i * size, i * size + size)
  );
}
```

For 10,000 Employees

```
10,000 tokens ÷ 500 per call = 20 FCM API calls
Takes ~2 seconds total
Run via Bull Queue so admin UI doesn't freeze waiting
```

---

13. Rate Limiting — Prevent Notification Spam

A bug in your code could trigger 1,000 notifications to one employee. Redis cooldown prevents this with zero extra infrastructure.

Redis Cooldown Per Employee Per Type

```javascript
async function sendPushNotification(emp_id, type, payload) {

  // Check cooldown in Redis
  const cooldownKey = `notif_cooldown:${emp_id}:${type}`;
  const alreadySent = await redis.get(cooldownKey);

  if (alreadySent) {
    console.log(`Cooldown active for ${emp_id} - ${type}. Skipping.`);
    return;
  }

  // Set cooldown BEFORE sending (not after — avoids race conditions)
  const cooldowns = {
    checkout_reminder: 60 * 60,    // 1 hour — no point resending before shift ends
    late_marked: 4 * 60 * 60,      // 4 hours — don't spam for same day
    leave_approved: 5 * 60,        // 5 minutes
    announcements: 10 * 60,        // 10 minutes
  };

  await redis.setex(cooldownKey, cooldowns[type] || 300, '1');

  // Now proceed with FCM send
}
```

Uses your existing Redis instance. Zero extra cost.

---

14. Offline Handling — Employee Has No Internet

What FCM Does Automatically

FCM holds undelivered notifications for **28 days** by default. When the employee comes back online, it delivers automatically. You do nothing extra.

```mermaid
```

```
sequenceDiagram
  participant BE as Your Backend
  participant FCM as Firebase FCM
  participant Phone as Employee Phone

  BE->>FCM: Send notification
  FCM->>Phone: Attempt delivery — FAILED (offline)
  FCM->>FCM: Hold notification in queue (up to 28 days)

  Note over Phone: Employee offline for hours...

  Phone->>FCM: Employee comes back online
  FCM->>Phone: Deliver held notification
```

The Edge Case — Stale Notifications

The checkout reminder was sent at 5 PM. Employee was offline. They come online at 9 PM. The notification arrives — but checkout time has long passed. Handle this in the app:

```javascript
messaging().onMessage(async (remoteMessage) => {
  const { type, sent_at } = remoteMessage.data;

  // Check if notification is stale (older than 2 hours)
  const ageInHours = (Date.now() - Number(sent_at)) / (1000 * 60 * 60);

  if (type === 'checkout_reminder' && ageInHours > 2) {
    return; // Don't show — irrelevant now, ignore silently
  }

  showInAppBanner(remoteMessage);
});
```

Set FCM TTL for Time-Sensitive Notifications

Tell FCM to discard the notification if not delivered within 1 hour:

```javascript
```

```
await firebase.messaging().send({
  token: fcm_token,
  notification: { title, body },
  android: {
    ttl: 3600000 // 1 hour in ms — FCM discards if not delivered in time
  },
  apns: {
    headers: {
      'apns-expiration': String(Math.floor(Date.now() / 1000) + 3600)
    }
  }
});
```

For checkout reminders specifically — if not delivered within 1 hour, FCM discards it. No point reminding someone to check out at midnight.

---

15. Badge Count — Stays Accurate Always

The number on the app icon must match actual unread notifications at all times.

The Full Badge Flow

```mermaid
sequenceDiagram
    participant BE as Backend
    participant DB as notifications table
    participant SO as Socket.io
    participant App as React Native App
    participant Icon as App Icon Badge

    BE->>DB: INSERT notification (is_read: false)
    BE->>SO: emit 'new_notification' to employee socket room
    SO->>App: Real-time event pushed to open session
    App->>Icon: Increment badge count +1

    Note over App: Employee opens notification screen

    App->>BE: PATCH /notifications/read-all
    BE->>DB: UPDATE is_read = true WHERE emp_id = $1
    App->>Icon: Set badge count = 0
```

On App Launch — Always Fetch Fresh Count

```javascript
```

```javascript
// Every time app comes to foreground — resync badge from DB
AppState.addEventListener('change', async (state) => {
  if (state === 'active') {
    const { count } = await api.get('/notifications/unread-count');
    setBadgeCount(count);

    // Also update the actual app icon badge (iOS)
    await notifee.setBadgeCount(count);
  }
});
```

Backend Unread Count Query

```sql
-- Fast because of partial index on is_read = FALSE
SELECT COUNT(*)
FROM notifications
WHERE emp_id = $1
AND is_read = FALSE;
```

The partial index means PostgreSQL only scans unread rows — not the entire notifications table. At millions of rows this matters.

---

16. iOS — The One Big Gotcha

On iOS, if the app is **background or killed**, FCM routes through Apple's APNs. This requires additional setup that Android does not need.

What you need:

1. Apple Developer Account ($99/year — needed anyway for App Store)
2. APNs certificate uploaded to Firebase console (one-time setup, 5 minutes)
3. `content-available: 1` in FCM payload for silent background wakes

```javascript
```

```
await firebase.messaging().send({
  token: fcm_token,
  notification: { title, body },
  apns: {
    payload: {
      aps: {
        sound: 'default',
        'content-available': 1  // allows app to wake in background on iOS
      }
    }
  }
});
```

Without this, iOS employees won't get the checkout reminder when the app is closed. This is the single most important iOS-specific setup step for your entire notification system.

---

17. Database Tables

device_tokens Table

```sql
CREATE TABLE device_tokens (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID NOT NULL REFERENCES organisations(id),
  emp_id      UUID NOT NULL REFERENCES employees(id),
  fcm_token   VARCHAR(500) NOT NULL,
  platform    VARCHAR(10) CHECK (platform IN ('android', 'ios')),
  is_active   BOOLEAN DEFAULT TRUE,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW(),

  CONSTRAINT unique_emp_token UNIQUE (emp_id, fcm_token)
);

-- Only index active tokens — faster FCM lookup query
CREATE INDEX idx_device_tokens_active
ON device_tokens (emp_id)
WHERE is_active = TRUE;
```

Why `is_active` instead of DELETE? FCM returns 404 when a token is dead. You flip `is_active = false` on 404. Never hard delete — keeps audit trail of which devices an employee used.

notifications Table

```sql
CREATE TABLE notifications (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  org_id      UUID NOT NULL REFERENCES organisations(id),
  emp_id      UUID NOT NULL REFERENCES employees(id),
  type        VARCHAR(50) NOT NULL,
  title       VARCHAR(255) NOT NULL,
  body        TEXT,
  data        JSONB,
  action_url  VARCHAR(500),
  is_read     BOOLEAN DEFAULT FALSE,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
  updated_at  TIMESTAMPTZ DEFAULT NOW()
);

-- Partial index — only indexes unread rows
-- Makes badge count query fast even at millions of rows
CREATE INDEX idx_notifications_unread
ON notifications (emp_id)
WHERE is_read = FALSE;
```

---

18. API Endpoints

Method	Endpoint	What It Does
GET	/api/v1/notifications	Fetch employee's notification list (paginated)
PATCH	/api/v1/notifications/:id/read	Mark one notification as read
PATCH	/api/v1/notifications/read-all	Mark all notifications as read
GET	/api/v1/notifications/unread-count	Returns badge number for bell icon
POST	/api/v1/device-tokens	Register FCM token on login
DELETE	/api/v1/device-tokens	Deregister FCM token on logout
PATCH	/api/v1/employees/me/notification-preferences	Update notification settings

---

19. Folder Structure

```
attendease/
└── backend/
    ├── services/
    │   └── notification.service.js     ← sendPushNotification() lives here
    ├── queues/
    │   ├── checkout-reminder.queue.js
    │   └── absent-alert.queue.js
    ├── workers/
    │   ├── checkout-reminder.worker.js
    │   └── absent-alert.worker.js
    └── routes/
        └── notifications.routes.js     ← all notification API routes
```

Firebase SDK is initialized **once** in `app.js` using your Firebase service account JSON stored as an environment variable — never hardcoded in source code.

---

20. Complete Notification Trigger Map

Trigger	Who	Bull Queue?	FCM Type	Deep Link
Forgot checkout	Employee	✅ Delayed 8hrs	Single	`checkout/:id`
Leave approved	Employee	❌ Immediate	Single	`leave/:id`
Leave rejected	Employee	❌ Immediate	Single	`leave/:id`
Late marked	Employee	❌ Immediate	Single	`attendance`
Absent marked	Employee	❌ Immediate	Single	`attendance`
Regularisation approved	Employee	❌ Immediate	Single	`regularise/:id`
Announcement	All / Dept	✅ Bull Queue	Multicast 500/batch	`home`
New employee onboarded	Employee	❌ Immediate	Single	`profile`

---

21. Cost Breakdown

Component	Service	Cost
Push delivery	Firebase FCM	**$0 forever**
Real-time badge	Socket.io on EC2	**$0**
Delayed jobs	Bull Queue on Redis	**$0** (existing Redis)
Bulk multicast	FCM Multicast API	**$0**
Badge count	PostgreSQL query	**$0**
Offline hold (28 days)	FCM buffer	**$0**
Rate limiting	Redis `setex`	**$0**

Total notification system running cost = $0/month

The only indirect cost is Redis memory for Bull Queue jobs:

- 10,000 employees × 2 jobs/day = 20,000 active jobs
- Each Bull job ≈ 1KB in Redis
- **20,000 KB = ~20 MB** — completely negligible on your existing Redis instance

---

Next Topics to Cover: Folder Structure → API Endpoint Design → Face Recognition Flow → JWT + Auth Flow → Offline Sync Logic